

# Materi Fundamental: Object-Oriented Programming (OOP)

## 1. What is OOP? (Apa itu OOP?)

**Object-Oriented Programming (OOP)** adalah sebuah *programming paradigm* (paradigma pemrograman) yang berfokus pada penggunaan "**Objects**" (objek) dibandingkan *functions* atau *logic* yang berurutan.

*Basically*, OOP mencoba mensimulasikan dunia nyata ke dalam kode pemrograman. Di dunia nyata, semuanya adalah objek (contoh: mobil, kucing, akun bank). Setiap objek punya:

- **Attributes / Properties:** Ciri-ciri atau data (contoh: warna, merk, kecepatan).
  - **Methods / Behavior:** Apa yang bisa dilakukan objek tersebut (contoh: melaju, ngerem, belok).
- 

## 2. Class vs Object: The Core Concept

Banyak yang masih bingung bedanya *Class* dan *Object*. *Let's make it simple:*

- **Class:** Ini adalah *blueprint* atau cetakan. Class mendefinisikan *properties* dan *methods* apa saja yang akan dimiliki oleh sebuah objek, tapi dia belum berbentuk nyata.
- **Object:** Ini adalah *instance* (wujud nyata) dari *Class*.

💡 **Analogi:** **Class** = Desain arsitektur sebuah rumah (*blueprint* kertas). **Object** = Rumah sungguhan yang sudah dibangun dari desain tersebut. Kita bisa bikin banyak rumah (objects) dari satu desain (class) yang sama!

---

## 3. The 4 Pillars of OOP (4 Pilar Utama)

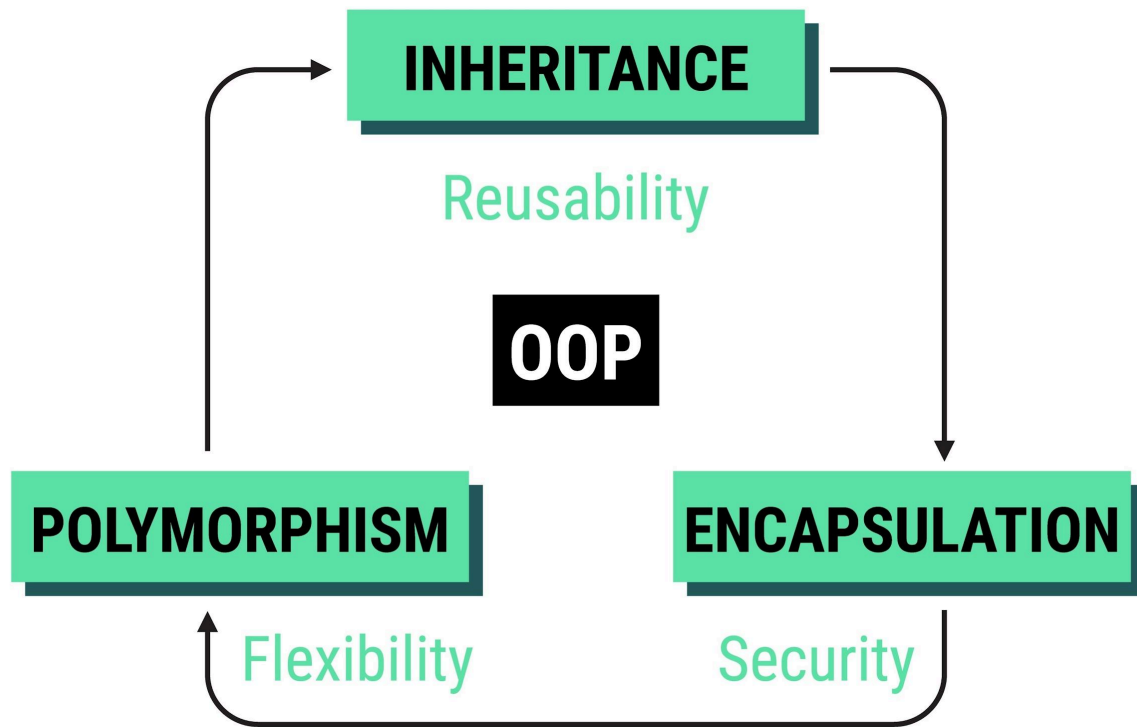
Untuk bisa dibilang menerapkan OOP yang baik, kita harus paham 4 pilar utama ini:

### A. Encapsulation (Enkapsulasi)

- **Concept:** *Data hiding* (menyembunyikan data). Kita membungkus data (*attributes*) dan fungsi (*methods*) ke dalam satu unit (yaitu Class), dan membatasi akses langsung dari luar.
- **Why it's useful:** Mencegah data diubah secara sembarangan oleh *unauthorized access*. Kita biasanya menggunakan *getter* dan *setter* methods untuk mengakses datanya.
- **Analogi:** Kayak kapsul obat. Kita tinggal minum kapsulnya untuk sembuh, tanpa perlu tahu atau menyentuh langsung bubuk kimia di dalamnya.

## B. Inheritance (Pewarisan)

- **Concept:** Sebuah *Class* baru (Child Class) bisa mewarisi semua *attributes* dan *methods* dari *Class* yang sudah ada (Parent Class).
- **Why it's useful: Code reusability!** Kita nggak perlu nulis ulang kode yang sama berkali-kali (DRY: *Don't Repeat Yourself*).
- **Analogi:** Seorang anak mewarisi ciri fisik dari orang tuanya (misal: warna mata, bentuk hidung), tapi si anak juga bisa punya keahlian sendiri yang nggak dimiliki orang tuanya.



Shutterstock

### C. Polymorphism (Polimorfisme)

- **Concept:** Berasal dari kata *poly* (banyak) dan *morph* (bentuk). Ini adalah kemampuan sebuah object atau method untuk mengambil banyak bentuk, tapi dieksekusi dengan cara yang berbeda tergantung object-nya.
- **Why it's useful:** Bikin kode lebih *flexible* dan *clean*.
- **Analogi:** Action "Bersuara". Kalau action ini dipanggil ke object Kucing, output-nya "Meong". Kalau dipanggil ke object Anjing, output-nya "Guk Guk". Nama method-nya sama, tapi *behavior*-nya beda.

## D. Abstraction (Abstraksi)

- **Concept:** Menyembunyikan *complex implementation details* (kerumitan di belakang layar) dan hanya menampilkan *essential features* (fitur-fitur penting) kepada user.
  - **Why it's useful:** Mengurangi kerumitan sistem dan memudahkan *user* saat menggunakan *object* tersebut.
  - **Analogi:** Saat kita menyetir mobil, kita cuma perlu tahu cara injak pedal gas untuk maju. Kita nggak perlu tahu gimana proses pembakaran bahan bakar di dalam mesin terjadi. *Just press the pedal and go!*
- 

## 4. Simple Code Example (Python)

Mari kita lihat implementasi *simple* OOP menggunakan Python:

Python

# Membuat Class

class Kucing:

# Constructor untuk inialisasi attributes

def \_\_init\_\_(self, nama, warna):

self.nama = nama

self.warna = warna

# Method / Behavior

def bersuara(self):

print(f"{self.nama} bilang: Meowww!")

# Membuat Object dari Class Kucing

kucing\_oren = Kucing("Oyen", "Oranye")

kucing\_hitam = Kucing("Blacky", "Hitam")

# Memanggil attributes dan methods

print(f"Kucing saya namanya {kucing\_oren.nama} dan warnanya {kucing\_oren.warna}.")

kucing\_oren.bersuara()